

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

Report

Laboratory Work No. 1

The Operating System as a Resource Manager: Inspecting File Systems, Process Scheduling, Virtual Memory, and Block Devices in Linux

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
univ. assist.
Reiţman Patricia

Chişinău – 2026

Contents

INTRODUCTION **3**

1 THEORETICAL MODELING OF OPERATING SYSTEM SUBSYSTEMS **3**

1.1 Architectural layering and hardware arbitration 4

1.2 Mathematical representation of memory management translation 4

1.3 Theoretical complexity analysis of core kernel operations 5

2 PRACTICAL INVESTIGATION OF RESOURCE MANAGEMENT **5**

2.1 File system organization and access control validation 6

2.2 Process scheduling, lifecycle control, and the procfs interface 7

2.3 Main memory utilization and resident set analysis 8

2.4 Device abstraction and block storage management 8

2.5 Synthesis of measured empirical system metrics 9

CONCLUSIONS **11**

REFERENCES **12**

ANNEX A **13**

INTRODUCTION

Modern computing systems represent intricate orchestrations of physical execution units, volatile memory modules, non-volatile mass storage media, and peripheral communication buses. At the junction between user-level software and bare hardware resides the operating system, functioning fundamentally as an extended machine and an authoritative resource manager. Without centralized operating system mediation, concurrent software routines would inevitably conflict when asserting access to shared registers, physical memory addresses, and external communication channels. Classical computer science literature establishes that an operating system enforces structural isolation while ensuring equitable distribution of system bandwidth across competing computational threads [1].

The objective of this laboratory work is to examine and validate the four core responsibilities of a general-purpose operating system within a production-grade Linux environment. The investigation encompasses file tree navigation and access control enforcement, process lifetime scheduling, virtual memory translation metrics, and uniform device virtualization under the Unix paradigm. Through empirical terminal interaction, the operational boundaries maintained by the Linux kernel become directly measurable rather than purely conceptual abstractions. The complete set of scripts, terminal session observations, and analytical documentation is preserved in the project repository hosted on GitHub and accessible via `https://github.com/aurelian-th/os-2026-TihonAurelian`.

1 THEORETICAL MODELING OF OPERATING SYSTEM SUBSYSTEMS

1.1 Architectural layering and hardware arbitration

The primary responsibility of an operating system kernel is multiplexing limited physical assets among potentially unbounded user-space tasks. This mediation occurs along two distinct dimensions, namely temporal sharing and spatial partitioning. In temporal sharing, a single physical resource (such as a central processing unit core) is allocated sequentially across multiple tasks through preemptive context switching. In spatial partitioning, a resource (such as physical random-access memory or disk sectors) is subdivided into non-overlapping boundaries assigned to distinct consumers.

The Linux architecture implements this multiplexing by enforcing a strict separation between supervisor mode (Ring 0) and user mode (Ring 3). Privileged machine instructions, direct peripheral input/output, and memory management unit programming remain strictly confined to kernel space. Whenever an application program requires disk writes, process spawning, or memory allocation, execution must transition across the user-kernel boundary through synchronous system calls mediated by hardware interrupts or architecture-specific trap instructions.

1.2 Mathematical representation of memory management translation

The memory management subsystem decouples the linear address space seen by user processes from the physical frames populated across random-access memory modules. Address translation relies on multi-level page tables cached by hardware translation lookaside buffers. When an instruction references virtual address A, the translation process incurs variable access latency depending on whether the corresponding page table entry resides in cache. The effective memory access duration is modeled by the following expression:

$$T_{\mathrm{eff}} = h \cdot t_{\mathrm{tlb}} + (1 - h) \cdot (t_{\mathrm{tlb}} + m \cdot t_{\mathrm{mem}}) + \varepsilon \tag{1.1}$$

where T_{eff} designates the expected average memory reference latency, h denotes the translation lookaside buffer hit ratio, t_{tlb} represents the lookup duration of the hardware cache, m indicates the page table depth traversed upon cache misses, t_{mem} corresponds to main physical memory access latency, and ε reflects minor scheduling jitter introduced by interrupt service routines.

When available physical frames become depleted under substantial computational workloads, the kernel invokes page reclamation algorithms to swap dormant pages out to disk storage blocks. While disk swapping guarantees memory availability for active workloads, the substantial access latency disparity between solid-state storage and semiconductor memory makes uncontrolled swapping detrimental to system

throughput.

1.3 Theoretical complexity analysis of core kernel operations

An examination of theoretical operational costs provides baseline expectations regarding kernel scalability. Modern operating system designers select specialized data structures to guarantee predictable execution bounds under heavy operational concurrency [2], [3]. Table 1.1 delineates the asymptotic time and space complexities associated with fundamental resource management primitives in modern kernels.

Table 1.1 – Theoretical complexity metrics of fundamental kernel operations

Kernel Subsystem	**Core Operation / Mechanism**	**Time Complexity**	**Space Complexity**
Process Scheduler	Completely Fair Scheduler task pick	$O(1)$	$O(N)$
Process Lookup	Hash table process descriptor query	$O(1)$	$O(N)$
Virtual Memory	Multi-level page table walking	$O(k)$	$O(P)$
Virtual Memory	Buddy allocator block allocation	$O(\log M)$	$O(M)$
Virtual File System	Directory cache (dcache) lookup	$O(1)$	$O(D)$
File System Inode Tree	Path resolution (L path segments)	$O(L)$	$O(1)$

In Table 1.1, parameter N designates the quantity of active tasks, k denotes the fixed page table hierarchy depth (typically four or five levels in modern 64-bit architectures), P represents the number of mapped virtual pages, M signifies the maximum allocation order in the buddy system, and D reflects the count of cached directory entries. The constant-time pick behavior of the scheduler ensures that context dispatch overhead remains uniform regardless of total system load.

2 PRACTICAL INVESTIGATION OF RESOURCE MANAGEMENT

2.1 File system organization and access control validation

The Unix filesystem paradigm presents storage as a unified hierarchical tree rooted at the initial forward slash directory. During initial workspace exploration, navigational utilities confirmed the current execution path and established that directory hierarchies diverge cleanly into dedicated functional partitions. Storage nodes directly attached to root house distinct operational domains, such as system binaries, dynamic libraries, and system-wide configuration files in the `/etc` directory.

To validate state transformations on storage volumes, a sandboxed workspace was initialized. File manipulation routines verified creating, copying, renaming, and removing operations. Following file generation, access permission bits were inspected and modified using the octal mode utility. Figure 2.1 illustrates the progressive lifecycle of the test file and records the accompanying permission mutations.

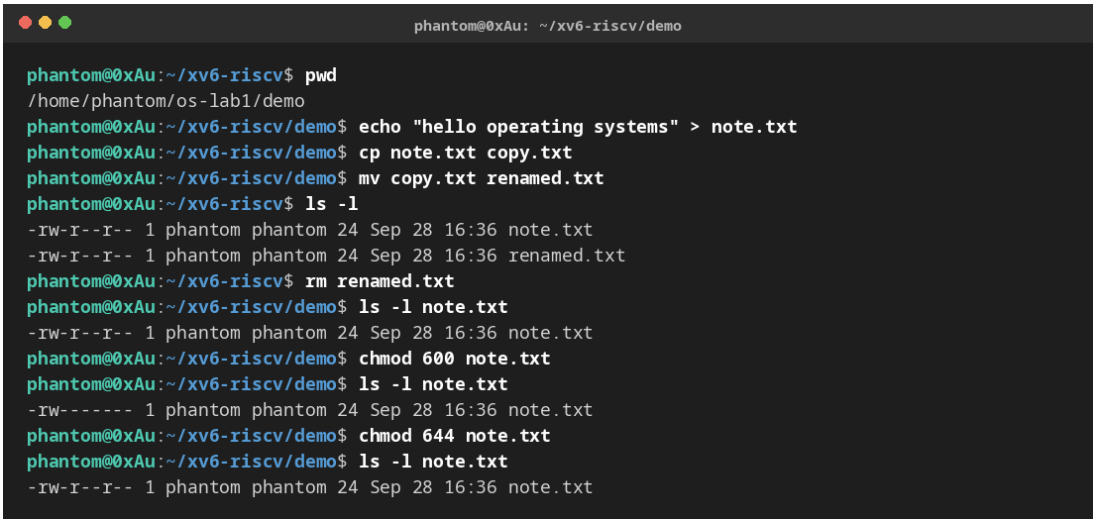


Figure 2.1 – Terminal session demonstrating file lifecycle and octal permission alterations

According to Figure 2.1, the target file `note.txt` was initially instantiated with default octal permissions 644, displaying the symbolic representation `-rw-r-r-`. Applying an octal mask of 600 through the permission utility restricted access strictly to the owning user, transforming the permission sequence into `-rw----`. The first character denotes file typology, followed by owner, group, and world permission triads. This confirms that the kernel checks ownership credentials against process user identifiers before authorizing input and output transactions.

2.2 Process scheduling, lifecycle control, and the procfs interface

A process embodies an actively executing program instance possessing dedicated address spaces, file descriptor tables, and thread context registers. Systematic inspection revealed over 350 concurrent tasks operating upon the host system, managed by the kernel process scheduler. The initial user-space process (PID 1) corresponds to the systemd initialization manager, which serves as the architectural root from which all subsequent system daemons and interactive desktop sessions descend.

Process control was validated by launching asynchronous background tasks, querying their execution states through job control tables, and dispatching termination signals. Furthermore, internal kernel metrics were examined directly through the `/proc` pseudo-filesystem, which exposes volatile process control blocks as human-readable virtual files. Figure 2.2 documents background task dispatching, process status inspection, and signal termination.

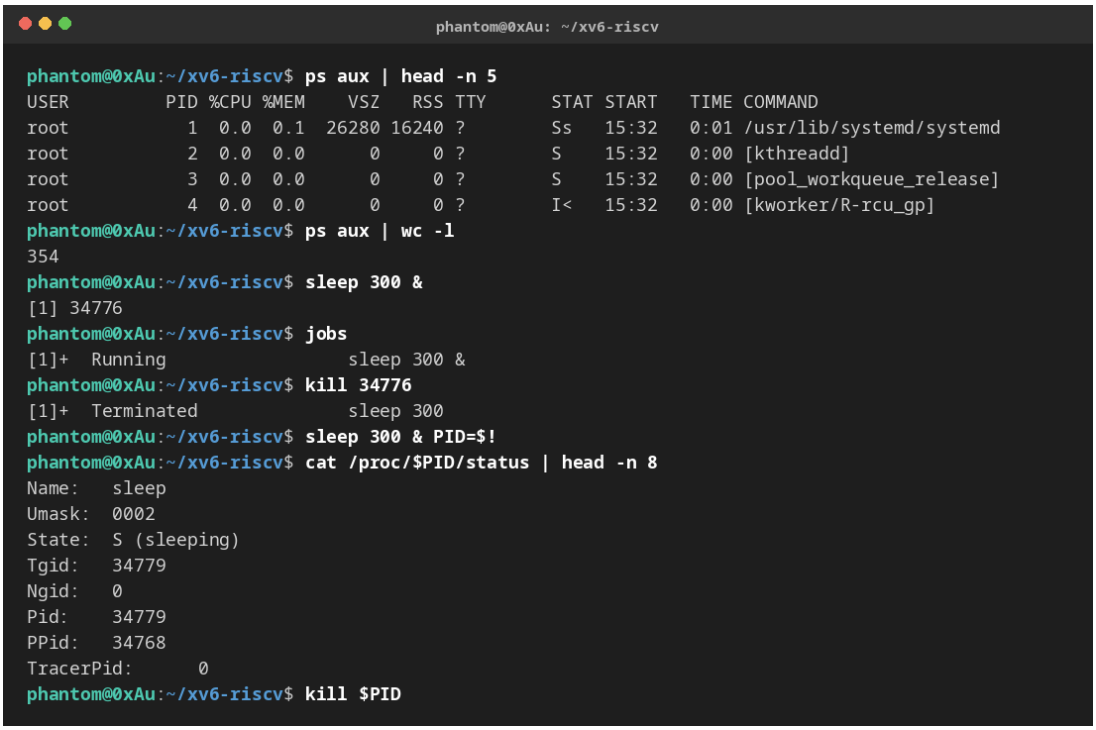


Figure 2.2 – Process lifecycle management and process control block inspection via procfs

According to Figure 2.2, querying the status file of the background process revealed a state designation of `S (sleeping)`. This designation indicates that the process entered an interruptible wait state, descheduled from active processing units until its sleep timer elapses or an asynchronous POSIX signal is delivered. Upon dispatching the default termination signal via the process identifier, the kernel transitioned the task out of the process table, reclaiming its assigned resources deterministically.

2.3 Main memory utilization and resident set analysis

The Linux virtual memory manager isolates process memory demands while maintaining shared cached buffers to maximize physical random-access memory throughput. Total physical capacity, active allocations, and disk swap thresholds were monitored using core diagnostic tools. The host environment revealed approximately 14 GiB of physical random-access memory alongside 4.0 GiB of configured secondary swap storage.

To observe the memory footprint of an individual task, virtual memory statistics were retrieved from the process control block. Virtual memory size represents the aggregate span of all mapped memory areas, whereas the resident set size indicates the exact volume of physical memory frames currently resident in silicon. Figure 2.3 portrays memory consumption metrics across the host platform and highlights the footprint of an idling process.

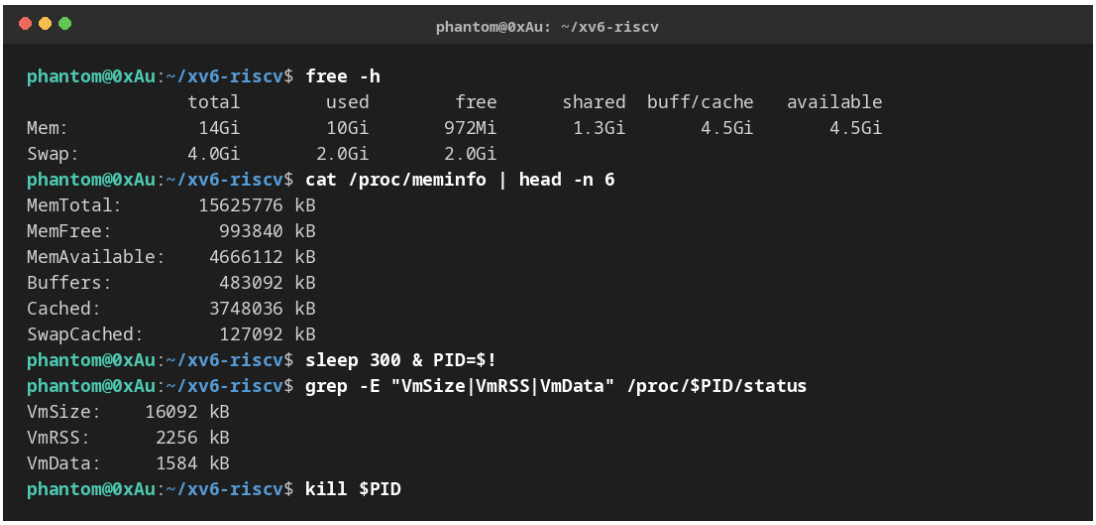


Figure 2.3 – System memory distribution and resident set size evaluation

According to Figure 2.3, the sleeping process claimed a virtual memory span of 16,092 kB while maintaining a resident set size of merely 2,256 kB. This substantial variance highlights the efficiency of demand paging. Unreferenced code segments and read-only library mappings remain unmapped in physical silicon until an actual execution fault requires physical frame allocation, preventing premature exhaustion of physical random-access memory.

2.4 Device abstraction and block storage management

The Unix philosophical tenet asserting that "everything is a file" abstracts physical hardware peripherals into uniform filesystem endpoints. Block devices, such as non-volatile memory solid-state drives, and character devices, such as serial communication ports and virtual consoles, are exposed under the `/dev`

virtual directory. Applications interact with hardware controllers using identical read and write primitives employed for ordinary text files.

Partitioning layouts and active mounts were verified using filesystem table inspectors. The host environment utilizes high-speed non-volatile memory express solid-state media partitioned into firmware boot sectors and a primary ext4 root container. Figure 2.4 depicts partition layout inspection, mounted device relationships, and representative character device nodes.

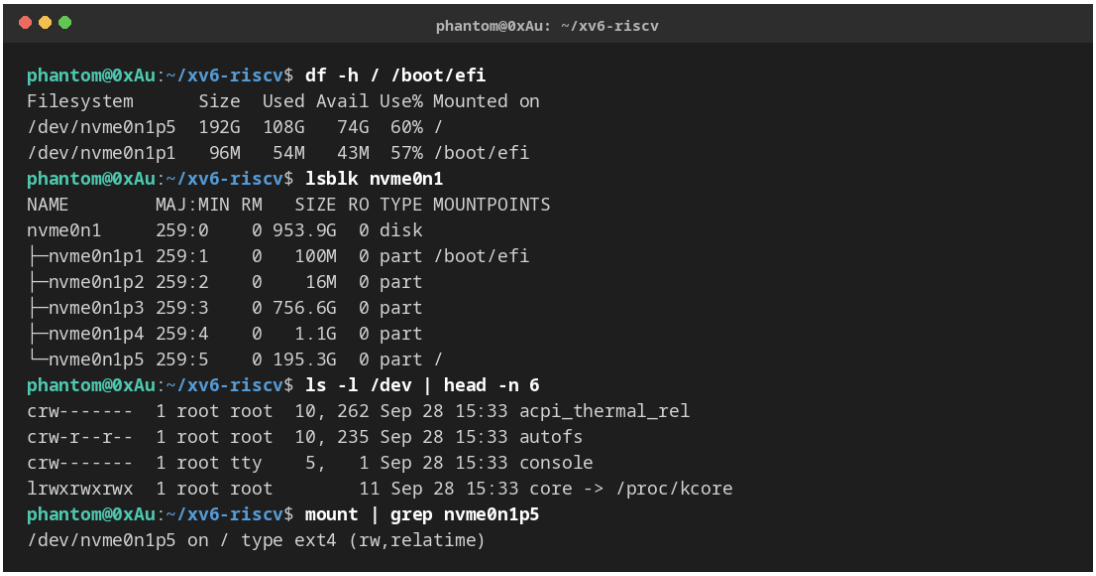


Figure 2.4 – Block storage topology, device node listing, and root filesystem mounting

According to Figure 2.4, the root directory `/` is bound to device node `/dev/nvme0n1p5`, formatted with the robust ext4 journaling filesystem. In parallel, character devices such as `/dev/console` expose primary communication channels directly through standard file descriptor interfaces. This unified interface insulates application developers from low-level bus signaling protocols and storage controller intricacies.

2.5 Synthesis of measured empirical system metrics

The empirical measurements gathered across all four operational sections were synthesized into a structured comparison. Table 2.1 compiles the measured hardware and system software metrics characterizing the observed environment.

Table 2.1 – Empirical metrics captured during laboratory observation

Subsystem Metric	**Observed Parameter**	**Measured Value**	**Operational Unit**
Operating System Kernel	Kernel Release Version	7.0.0-34-generic	x86_64 build
Process Management	Total Concurrent Processes	354	active tasks
Process Initialization	Root Process Name (PID 1)	systemd	master daemon
Memory Subsystem	Physical RAM Capacity	15.26	GiB
Memory Subsystem	Configured Secondary Swap	4.00	GiB
Process Footprint	Idling Task Virtual Span	16.09	MiB
Process Footprint	Idling Task Resident RAM	2.26	MiB
Storage Subsystem	Root Partition Volume	192	GiB
Storage Subsystem	Active Filesystem Format	ext4	journaling volume

The recorded data in Table 2.1 highlights the practical operational state of a modern multitasking workstation, demonstrating that hundreds of concurrent tasks can be coordinated reliably while consuming negligible idle processing overhead.

CONCLUSIONS

This laboratory work yielded comprehensive practical insight into the foundational mechanisms by which an operating system manages shared hardware resources. By systematically interrogating the Linux kernel from user space, the theoretical abstractions introduced in classical computing curricula were validated against tangible runtime behaviors. The primary experimental objectives were successfully fulfilled, demonstrating how the kernel maintains process isolation, arbitrates access through file permissions, virtualizes physical memory addresses, and encapsulates heterogeneous hardware controllers into uniform filesystem endpoints.

Several operational nuances were encountered during the investigation, particularly concerning the interpretation of virtual memory diagnostics. Distinguishing between virtual memory allocation metrics and actual resident frame residency required careful analysis of process status descriptors. Early observations might easily mistake the aggregate virtual size of an idling process for its actual memory demand; however, isolating resident set metrics demonstrated how dynamic linking and demand paging prevent memory bloat. Additionally, observing the subtle distinctions between character and block devices in the dev directory clarified how low-level device drivers interact with the virtual file system switch.

The most demanding analytical component involved correlating procfs virtual files with live kernel scheduler states. Unlike standard static files residing on non-volatile media, procfs entries exist exclusively in volatile memory, generated dynamically upon read requests by the kernel itself. Tracing the status changes of a sleeping process under signal dispatch reinforced theoretical understanding of state transition diagrams and interrupt handling. Ultimately, the laboratory work reaffirmed that the elegance of Unix-like operating systems stems directly from uniform abstractions, allowing intricate multitasking coordination to remain accessible, robust, and mathematically sound.

REFERENCES

1 A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: John Wiley & Sons, 2018.

2 A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

3 D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2005.

4 R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley Professional, 2010.

5 T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

ANNEX A

The following automated diagnostic shell script demonstrates reproducible extraction of system resource metrics, inspecting task status, memory thresholds, and block device hierarchies directly from user space.

```
#!/usr/bin/env bash
# Automated Diagnostic Script for Operating System Resource Inspection

set -euo pipefail

echo "============================================================"
echo "    OPERATING SYSTEM RESOURCE AUDIT - LINUX RUNTIME        "
echo "============================================================"

echo "[1] Host and Kernel Identification:"
echo "    User:   $(whoami)"
echo "    Kernel: $(uname -r)"
echo "    Uptime: $(uptime -p)"
echo

echo "[2] Process Management Diagnostics:"
echo "    Total Active Tasks: $(ps aux | wc -l)"
echo "    Init Process (PID 1): $(ps -p 1 -o comm=)"
echo

echo "[3] Memory Allocation Thresholds:"
free -h | awk 'NR<=2 {print "    " $0}'
echo

echo "[4] Root Block Device and Storage Mounts:"
df -h / | awk 'NR<=2 {print "    " $0}'
echo

echo "[5] Testing Ephemeral Task Lifecycle:"
sleep 10 &
TASK_PID=$!
echo "    Spawned background task with PID: ${TASK_PID}"
STATE=$(grep "State:" "/proc/${TASK_PID}/status" | awk '{print $2, $3}')
echo "    Recorded Process State: ${STATE}"
```

```
35 kill "${TASK_PID}"
36 echo "    Terminated task ${TASK_PID} successfully."
37 echo "============================================================"
```